Introduction to Python

~ G. Anushiya Rachel

Assistant Professor, CSE

Shiv Nadar University Chennai

Introduction

- Created by Dutch programmer, Guido van Rossum
- First released in the early 1990s
- Name comes from a 1970s British comedy show, “Monty Python’s Flying Circus”

Why Python?

- Simple syntax
- Programs are clear and easy to read
- Provides powerful programming features and is widely used
- Well-supported and freely available

Python Shell – Interactive Mode

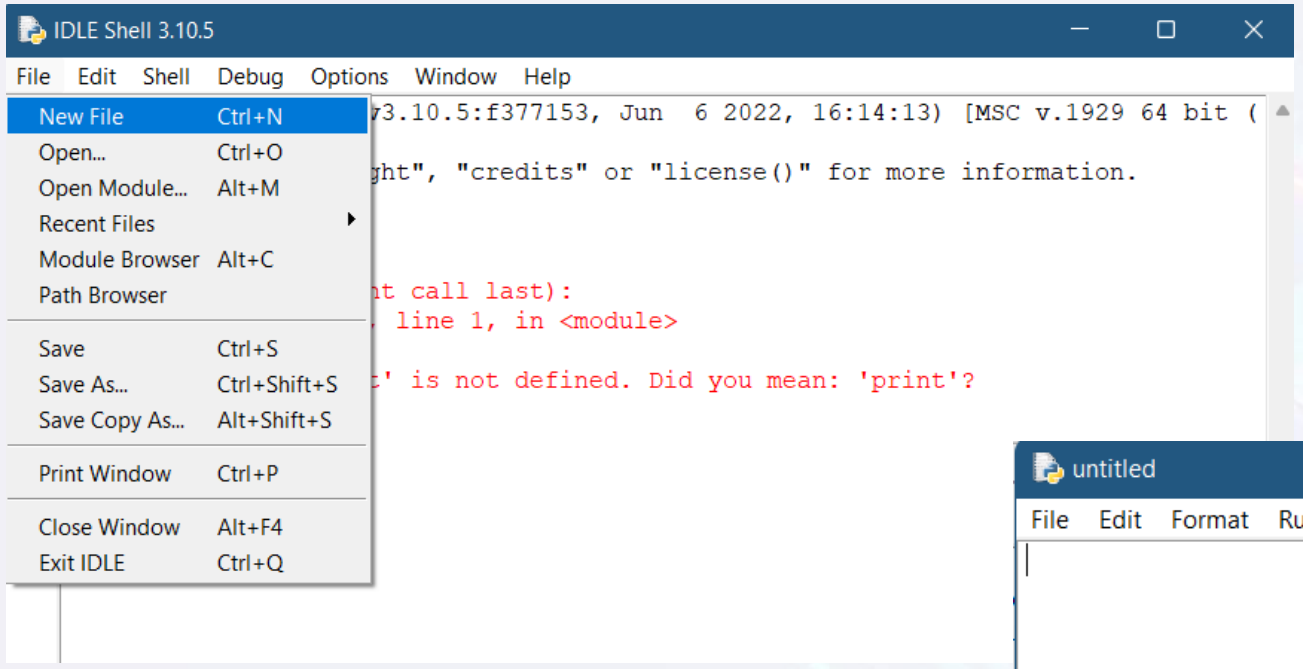
IDLE Shell 3.10.5

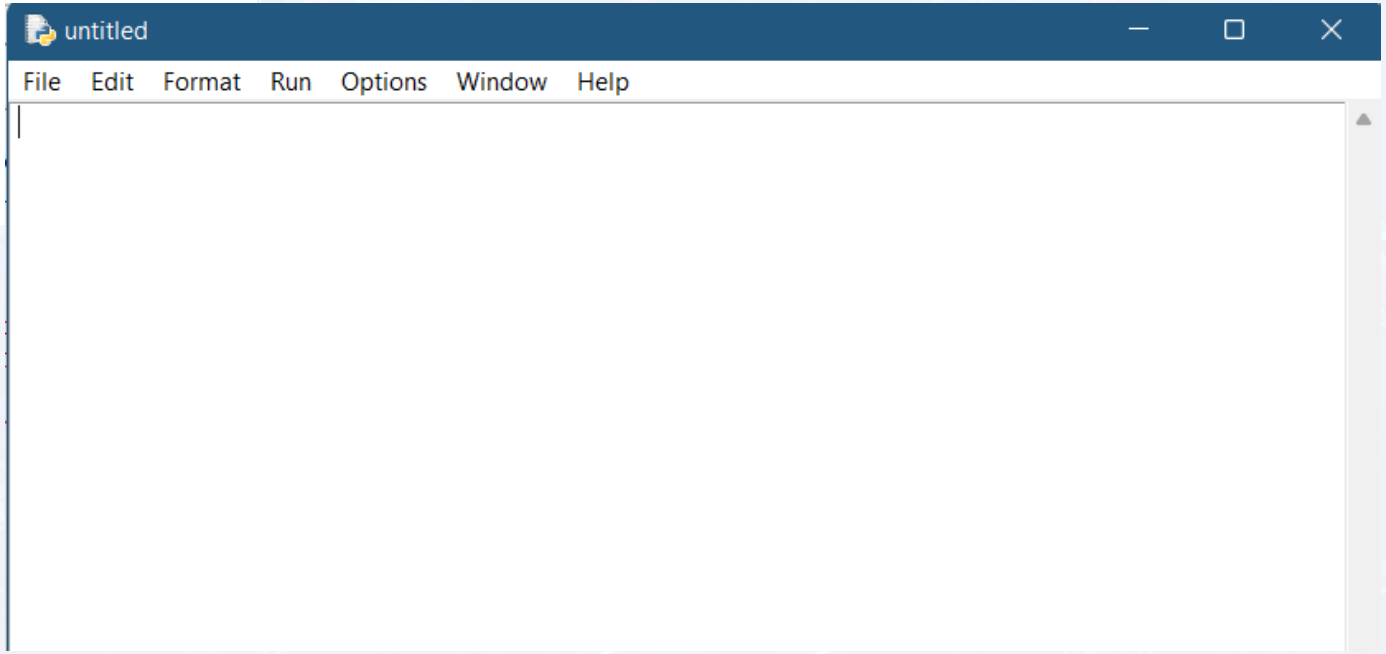
File Edit Shell Debug Options Window Help

```
Python 3.10.5 (tags/v3.10.5:f377153, Jun  6 2022, 16:14:13) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> 2 + 3
5
>>> prnt(2)
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    prnt(2)
NameError: name 'prnt' is not defined. Did you mean: 'print'?
>>> print(2)
2
>>>
```

Shell prompt

Program Editor – Script Mode





Major Differences From C

- C programs are compiled, unlike Python programs which are interpreted.
- C requires the variable type to be declared, while Python does not.
- All statements in C are punctuated with a semicolon, except loop/conditional statements, function declaration. This is not done in Python.
- Python requires the use of indentations to denote a block of code, unlike C which uses brackets instead.

Literals

- Sequence of 1 or more characteristics that stands for itself
  - Numeric literal
    - Contains only digits 0 to 9, an optional sign character (+ or –), decimal point
    - Commas not allowed
  - String literal
    - Sequence of characters enclosed in quotes – single (‘ ’) or double (“ ”)

'Jennifer Smith's Friend' ... *matching quote?*

'A'	- a string consisting of a single character
'jsmith16@mycollege.edu'	- a string containing non-letter characters
"Jennifer Smith's Friend"	- a string containing a single quote character
' '	- a string containing a single blank character
''	- the empty string

Literals

LET'S TRY IT

From the Python Shell, enter the following and observe the results.

```
>>> 1024
???

>>> 1,024
???

>>> -1024
???

>>> 0.1024
???

>>> .1024
???

>>> 1,024.46
???
```

LET'S TRY IT

From the Python Shell, enter the following and observe the results.

```
>>> print('Hello')
???

>>> print("Hello")
???

>>> print("Hello")
???

>>> print("Let's Go!')
???

>>> print('Let's Go')
???

>>> print("Let's go!")
???
```

Control Characters

- Not displayed on the screen
- Control the display of output
- Represented by a sequence of characters called escape sequence
  - Begins with an escape character (\) – causes the sequence of characters following it to "escape" their normal meaning
  - Example: \n – newline control character

LET'S TRY IT

From the Python Shell, enter the following and observe the results.

```
>>> print('Hello World')
???

>>> print('Hello World\n')
???

>>> print('Hello World\n\n')
???

>>> print('\nHello World')
???

>>> print('Hello\nWorld')
???

>>> print('Hello\n\nWorld')
???

>>> print(1, '\n', 2, '\n', 3)
???

>>> print('\n', 1, '\n', 2, '\n', 3)
???
```

Variable Names / Identifiers

- Case sensitive – *line* is different from *Line*
- May contain letters and digits and underscore (_) as well
- Must begin with a letter
- Must not be keywords

Valid Identifiers	Invalid Identifiers	Reason Invalid
totalSales	'totalSales'	quotes not allowed
totalsales	total sales	spaces not allowed
salesFor2010	2010Sales	cannot begin with a digit
sales_for_2010	_2010Sales	should not begin with an underscore

Keywords

- Identifier that has a predefined meaning
- Using keywords as a “regular” identifier will result in a syntax error.

and	as	assert	break	class	continue	def
del	elif	else	except	finally	for	from
global	if	import	in	is	lambda	nonlocal
not	or	pass	raise	return	try	while
with	yield	false	none	true		

Comments

- Not executed, Used for documentation purposes to explain what the program is doing
- Single line comments begin with #

I/O Statements

- input

```
>>> var = input ('prompt')
>>> name = input('Enter your name: ')
>>> num = int(input('Enter a number: '))
```

- print

```
>>> print("Hello")
>>> print('c = ', 30)
>>> print('c = ', c)
```

  - The print statement usually terminates in a newline. In order to terminate in a different character, use the *end* option.
    - *print*('c = ', c, *end* = ' ') # Terminates in a space

The value read by the *input* function will always be a string. When dealing with numbers, perform type conversion.

Importing Modules

- import module
  - $import\ math$
  - Access functions as $math.exp$, $math.sin$, etc.
- from module import function
  - $from\ math\ import\ exp$
  - $from\ math\ import\ pi$
  - Access functions/constants as exp, pi, etc.
- import module as renamed module
  - $import\ math\ as\ m$
  - Access functions/constants as $m.exp$, $m.sin$, $m.pi$, etc.

Data Types

- Numeric – int, float, complex (4+5j or complex(4,5))
- String – str
- Boolean – bool – True or False
- Sequence – list, tuple, range
  - List: [1, 2, 4, ‘hello’]
  - Tuple: (1, 2, 4, ‘hello’)
  - Range: range(1, 10, 2) # range(start, stop, step)
- Dictionary – dict – mapping
  - {‘Jan’:1, “Feb”:2} # {key1 : value1, key2 : value2}
- Set – set
  - {1, 2, ‘s’}

Check Type and Type Conversion

- To check the data type, use the $type$ function.
  - $a = 10$
  - $type(a)$ # Displays int
- To convert from one type to another (Explicit Type Conversion)
  - $b = float(a)$
  - $d = int(b)$
  - $c = complex(a, b)$ # Note: To get the real and imaginary parts use $c.real$ and $c.imag$
  - $l = [1, 2, 3]$
  - $tuple(l)$
  - $str(b)$

Implicit conversion – Automatic conversion when dealing with expressions

Arithmetic Operators

- Addition +
- Subtraction –
- Multiplication *
- Division / Always results in a floating-point number
- Floor division // When any one operand is of type float, the output is too
- Modulus %
- Exponential **

Rules of Precedence

- Parentheses ()
  - Exponentiation **
    - Multiplication *
    - Division /
    - Truncation division //
    - Modulus %
      - Addition +
      - Subtraction –

PEMDAS

- Examples:

```
>>> 1 + 2 * 3          7

>>> (1 + 2) * 3        9

>>> 4 – 40 – 3         -39

>>> 4 – (40 – 3)       -33
```

Arithmetic Overflow and Underflow

- Floating point numbers
  - Limits of range in floating point numbers
    - Range of 10^{-308} to 10^{308} with 16 to 17 digits of precision
  - Can be represented in scientific notation

`9.0045602e+5`	(9.0045602×10^5, 8 digits of precision)
`1.006249505236801e8`	($1.006249505236801 \times 10^8$, 16 digits of precision)
`4.239e−16`	(4.239×10^{-16}, 4 digits of precision)

  - Sometimes certain calculations can produce a result that is **too large** or **too small** to be represented → **arithmetic overflow** and **arithmetic underflow**
    - Results in infinity or 0 respectively

format Function

- Since floating point values may contain an arbitrary number of decimal places, this function can be used to produce a *string* version of the number with the specified number of decimal places.
- Examples:

```
>>> 12/5
2.4
>>> format(12/5, '.2f')
'2.40'
```

Format specifier

```
>>> 5/7
0.7142857142857143
>>> format(5/7, '.2f')
'0.71'
```

- For every large or very small values, *e* can be used as a format specifier.

```
>>> format(2 ** 100, '.6e')
'1.267651e+30'
```

Other Operators

- Assignment
  - $=,\ +=,\ *=,\ -=,\ /=,\ \%=,\ **=$
  - $a += 10 \rightarrow a = a + 10$
- Logical
  - and, or, not
- Relational
  - $==,\ !=, >, <,\ >=, <=,$
- Bitwise
  - $\&, |, \sim. \wedge, \gg, \ll$
- Membership – in

Conditional Statements

- Indentation is important when dealing with conditional and iteration statements.
- $if - else$
  - $if\ condition$:

    $statements$

    $else$:

    $statements$
- $if - elif - else$
  - $if\ condition$:

    $statements$

    $elif\ condition$:

    $statements$

    $else$:

    $statements$
- Equivalent of switch-case in C → $match - case$
  - Available in Python 3.10 and above

Iteration

- for loop
  - $for\ val\ in\ sequence$:

    $statements$
  - Example:
    - $a = 5$

    $for\ i\ in\ range(5)$:

    $a = a + 1$
- $while$ loop
  - $while\ condition$:

    $statements$
- $break$ and $continue$ – $break$ terminates the loop, and $continue$ skips an iteration

$range(N)$ lists numbers from 0 to $N - 1$.
To list numbers in descending order:
$range(N, 0, -1)$ # numbers from N to 1.

Some Built-In Functions

- Seen earlier – $input, print, format$, $complex$, type conversion ($int, str, list$, etc.),
- $abs(x)$ – Gives the absolute value / magnitude of an integer or a floating-point number or a complex number
- $eval(expression)$ – Evaluates the specified expression (which should be a string). If the expression is a valid Python statement, it is executed.
- $hex(x), oct(x), bin(x)$ – Converts an integer, x, into hexadecimal, octal, or binary value respectively
- $round(x, num_digits)$ – Rounds off a number, x, to num_digits decimal places (If num_digits is not mentioned, the number is rounded off to an integer.)

Practice Programs

- Write Python programs for the following:
  1. Prompt the user to enter three numbers and calculate the sum, mean, maximum and minimum values and display the results.
  2. Obtain the mark scored by a user out of 100, as an integer and, categorize it as follows:

90 <= mark <= 100	outstanding
80 <= mark < 90	excellent
70 <= mark < 80	good
60 <= mark < 70	average
50 <= mark < 60	below average
mark < 50	poor

Practice Programs

3. Get the temperature in Celsius from the user and convert it to Fahrenheit.
4. A company pays its employees at an hourly rate and provides an overtime pay that is 1.5 times the hourly rate, for every additional hour, if an employee has worked over 40 hours. Calculate the pay earned by an employee, given the number of hours he has worked. Round off the pay to 2 decimal places.
5. Generate the Fibonacci series up to N. Obtain the value of N from the user.
6. Display the odd numbers among N integers obtained from the user. Display the odd numbers in the same line, separated by a comma.
7. Print all numbers between 1 and N (specified by the user) which when divided by a number, M, specified by the user will result in a remainder of 5. Count the number of values that satisfy the condition and compute their average and print it.

Practice Programs

8. Print all the prime numbers up to a number N given by the user.
9. Write a program to print Floyd's triangle with the number of rows specified by the user.
   Floyd's triangle with 4 rows is as follows:
   1
   2 3
   4 5 6
   7 8 9 10
10. Write a program to print Pascal's triangle with the number of rows specified by the user.
   Pascal's triangle with 6 rows is as follows:

```
        1
      1   1
    1   2   1
  1   3   3   1
1   4   6   4   1
1   5   10  10  5   1
```

Practice Programs

11. Find the sum of the series, $x - x^3 + x^5 - x^7 + \cdots$ up to N terms. Obtain x and N from the user.
12. Print the following patterns up to N rows, where N is obtained from the user.

a)

```
        *
    *   *
*   *   *
*   *   *
```

b)

```
    *
  *   *
*   *   *
*   *   *
```

13. Get the real and imaginary parts of two complex numbers and construct the complex numbers. Check which arithmetic operations can be performed in the interactive mode. Find the magnitude and angle/phase of the complex numbers.

Practice Programs

14. Print a hollow square pattern where the border and the two main diagonals are filled with numbers 1 to N in each row, where N is the number of rows obtained from the user.
15. Find the sum of the series 1, (1+2), (1+2+3), (1+2+3+4), … (1+2+…+N), where N is the number of elements in the series, obtained from the user.
16. Print a diamond-shaped pattern with numbers from 1 to N. Obtain the N from the user.